QueueLess: Interview Questions and Answers

Every likely interview question about this project, with a clear answer, where it lives in the code, and how it is implemented.

For: Jayasri · 96 questions in 10 sections · Code references match the project on 2 November 2026 (if a line number has moved, search for the function name).

How to use this. Cover the answer and say your own answer out loud first, then compare. Interviewers rarely want the exact words here: they want to hear that you **understand why**. Open the file named under each answer and read the code around it. The questions most likely to become deep dives are marked in section 7 and section 5: token numbering, call next, refresh tokens and logout, and how the TV never sees names.

Contents

1. About the project (7)
2. Frontend (React) (10)
3. Backend (Node.js + Express) (10)
4. Database (MongoDB) (11)
5. Authentication and security (16)
6. Real-time (Socket.IO and Redis) (8)
7. Concurrency and system design (9)
8. Features and business logic (10)
9. Testing and debugging (9)
10. Behavioural and resume questions (6)

1. About the project

Almost every interview starts here. Answer in plain words first; details come later.

1. Explain QueueLess in 60 seconds.

QueueLess is a **virtual queue system** for places where people wait in line: clinics, banks, canteens, salons and offices. Instead of standing in a crowded room, a customer **scans a QR code** at the entrance, types their name and phone number, and gets a token (for example `A-012`) on their phone. The page then shows, **live**, how many people are ahead and the estimated wait, and alerts them when it is almost their turn.

Behind the counter, **staff** use a console to call the next person, mark them served or as a no-show. A **TV display** in the waiting area shows who is being served now. The **owner** has an admin panel to set up services, counters or doctors, staff accounts, opening hours, branding, and to see analytics.

Technically it is a full-stack app: a **React** frontend, a **Node.js/Express** REST API, **MongoDB** for data, **Redis** for rate limits, locks and real-time relay, and **Socket.IO** for live updates. It is multi-tenant: many businesses share one system, and each sees only its own data.

2. Why did you build this? What problem does it solve?

Waiting rooms are crowded, people cannot leave without losing their place, and staff spend time shouting names. QueueLess lets customers wait anywhere and come back when the phone says so, gives staff one button to call the next person, and gives owners numbers (busy hours, average wait, no-shows) to plan staffing. It needs no app install and no customer account, which is what makes adoption realistic for a clinic or a canteen.

For me it was also a way to practise the hard parts of real products: real-time updates, concurrency (two staff, one customer), security, and multi-tenancy.

3. Walk me through the architecture.

There are five parts:

- **Frontend** (React 18 + Vite, Tailwind, Zustand) running in the browser on port 5173.
- **Backend API** (Node 22 + Express 5) on port 4000, organised in layers: routes, middleware, controllers, services, models.
- **MongoDB** (Mongoose) for permanent data: businesses, users, tokens, invites, invoices.
- **Redis** for short-lived data: rate-limit counters, PIN lockouts, job locks, and the Socket.IO relay between servers.
- **Socket.IO** on the same port as the API for live events.

The browser calls the API over HTTP for every read and write, and keeps one Socket.IO connection open to hear "something changed". When a service changes data, it emits an event to a **room** (for example the TV room of that business); the screens in that room reload what they need. The browser never touches MongoDB or Redis directly.

In the code: backend/server.js (start-up), backend/app.js (middleware + routes), frontend/src/main.jsx and App.jsx

4. What was the hardest part?

Two things stand out. First, **concurrency**: two staff members pressing "Call next" at the same moment must never get the same customer, and two customers joining at the same moment must never get the same token number. I solved both with atomic database operations instead of "read, then write" (details in section 7).

Second, **sending live updates only to the right people**: the TV must show token codes but never customer names, a phone must only hear about its own token, and one business must never receive another's events. I solved that with Socket.IO rooms and a separate payload per audience.

5. What would you improve with more time?

- **Deploy it**: frontend on Vercel, backend on a host that keeps a long-running process (Render or Railway), MongoDB Atlas and a cloud Redis.
- **Cloud storage for logos** (S3 or Cloudinary) instead of the local disk.
- **A real SMS provider** (MSG91 or Twilio) for "almost your turn" texts; the code path exists but only logs today.
- **Frontend tests** (React Testing Library, Playwright); the backend has 247 tests, the frontend has none yet.
- **Better wait estimates** from real service times instead of a configured average (a comment in eta.service.js marks this).
- A **job queue** (BullMQ) instead of timers, for retries and visibility.

6. Is it deployed? How many users does it have?

Be honest: it runs locally (Docker for MongoDB and Redis, `npm run dev` for both apps), with demo data from a seed script. It has no real users. What proves it works is the **247 automated tests** and end-to-end checks in a real browser. Saying this confidently is better than inventing numbers; interviewers check.

7. What did you learn from this project?

Good themes to mention: designing an API in layers so business rules live in one place; how JWT access and refresh tokens work together; why "read then write" is unsafe under concurrency and how atomic updates, unique indexes and transactions fix it; how WebSocket rooms work; and the value of tests that run against a real (in-memory) database.

2. Frontend (React)

How the browser side is organised and how it talks to the server.

8. How is the frontend organised?

By responsibility:

- `pages/` : one file per screen (admin, staff, customer, display, auth, onboarding, marketing).
- `components/` : shared pieces (tables, dialogs, charts, the staff panels).
- `layouts/` : page frames and guards (`RequireRole`).
- `store/` : Zustand stores, a cache of server data plus UI settings.
- `hooks/` : reusable logic such as `useLiveQuery` and `usePublicBusiness` .
- `lib/` : `api.js` (all HTTP), `socket.js` (live connection), `fakeBackend.js` (the named server functions pages call).

`App.jsx` lists every route, so it is the table of contents of the frontend.

In the code: frontend/src/App.jsx:55 (routes)

9. Why Zustand rather than Redux or React Context?

Zustand gives a global store with almost no boilerplate (no actions, reducers or providers), components subscribe to just the slice they need so they re-render less than with one big Context, and it is easy to read and write from outside React (the API layer updates stores directly). The app's global state is small: who is logged in, the current business, today's queue, notifications, UI preferences. Redux would add ceremony without benefit at this size.

Important design point: the stores are a **cache of server data**, not the source of truth. Only UI preferences are saved in the browser.

In the code: frontend/src/store/authStore.js, businessStore.js, queueStore.js, uiStore.js

10. Where do all API calls go, and why one place?

Every request goes through `request()` in `lib/api.js`; there are only two `fetch` calls in the frontend. One place means: the login token is added the same way everywhere; an expired token is refreshed and the request retried automatically; every answer has the same shape (`{ ok, error, code, details }`) so pages never deal with raw HTTP; and dates are converted once. Pages never need try/catch around network calls.

```
export async function request(method, path, { body, form, query, raw = false, retry = true } =
{}) {
  const headers = {};
  if (accessToken) headers.Authorization = `Bearer ${accessToken}`;
  if (body !== undefined) headers['Content-Type'] = 'application/json';
  let res;
  try {
    res = await fetch(`${API_URL}${path}${qs}`, { method, headers, credentials: 'include',
      body: form ?? (body !== undefined ? JSON.stringify(body) : undefined) });
  } catch {
    return failure(0, { code: 'NETWORK', message: 'Could not reach the server. Check your
connection.' });
  }
  // An expired access token: refresh once, then repeat the request.
  if (res.status === 401 && retry && !path.startsWith('/api/auth/')) {
    const refreshed = await refreshSession();
    if (refreshed.ok) return request(method, path, { body, form, query, raw, retry: false });
    onSessionLost();
  }
  const data = await res.json().catch(() => ({}));
  if (!res.ok || data.ok === false) return failure(res.status, data.error);
  return revive(data);
}
```

In the code: frontend/src/lib/api.js:64

11. How do you protect admin and staff pages in the frontend?

`RequireRole` wraps the route. It waits for the start-up session check (showing a spinner), sends you to `/login?next=...` if you are not logged in, and to your own home if your role does not match (staff trying to open `/admin` go to `/staff`).

The key point for interviews: **this is only for user experience**. Real protection is on the server; every admin API route checks the token and role again. Anyone can bypass frontend code.

In the code: frontend/src/layouts/RequireRole.jsx:9; server side: backend/src/routes/admin.routes.js (requireAuth, requireOwner, tenantScope)

12. Why keep the access token in memory and not in localStorage?

Anything in localStorage can be read by any JavaScript on the page, so one XSS bug (or a bad browser extension) could steal it. A variable in memory disappears on reload and is harder to reach. The long-lived **refresh token** is in an **HTTP-only cookie**, which JavaScript cannot read at all. After a reload, the app calls `/api/auth/refresh`; the browser sends the cookie automatically and gets a fresh access token.

In the code: frontend/src/lib/api.js:9 (accessToken variable), backend/src/controllers/auth.controller.js:9 (cookie options)

13. How does the page stay logged in after a refresh?

`main.jsx` calls `restoreSession()` before any page needs data. It calls `POST /api/auth/refresh`; the HTTP-only `ql_refresh` cookie goes with it. If the server accepts it, a new access token comes back, the app loads `/api/auth/me` into the stores and reconnects Socket.IO. If not, the user sees the login page. Pages behind a login show a spinner until this check finishes, so they never flash the login page by mistake.

In the code: frontend/src/main.jsx:11, frontend/src/lib/fakeBackend.js:103 (restoreSession)

14. How do screens update without reloading the page?

The browser keeps one Socket.IO connection. When something changes on the server (a customer joins, a token is called), the server sends an event to the right room. The listening screens then **reload the data they show**: the staff console calls `loadQueue()`; Home, Customers and Analytics use the hook `useLiveQuery`, which reloads shortly after any live event; the token page and TV reload their own status.

In the code: frontend/src/lib/fakeBackend.js:144 (startLive), frontend/src/hooks/useLiveQuery.js:11

15. How does useLiveQuery avoid an old answer overwriting a newer one?

Several events can arrive close together, so several requests may be in flight. Each run takes a number from a counter; when its answer arrives, it is ignored unless it is still the newest. Without this, a slow old response could arrive last and show stale data.

```
const run = useCallback(async () => {
  const id = ++latest.current;
  const res = await load();
  if (id !== latest.current) return; // a newer request has started since
  if (res.ok) { setData(res); setError(null); } else setError(res.error);
  setLoading(false);
}, deps);
```

In the code: frontend/src/hooks/useLiveQuery.js:17

16. How do you validate forms, and why validate again on the server?

Forms use **react-hook-form** with **Zod** schemas, so users get instant messages next to each field. The server validates every request again with its own Zod schemas, because the frontend can be bypassed: anyone can send any request with Postman or curl. Frontend validation is for user experience; server validation is for correctness and security. Server errors come back with `details` per field, which the forms show next to the right input.

In the code: frontend/src/lib/validators.js; backend/src/validators/*.js

17. How does a customer's phone remember its token without an account?

When a customer joins, the server returns the token id and a random **guest secret** once. The page saves both in `localStorage` (`ql-guest-token:<slug>` and `ql-guest-secret:<tokenId>`). Coming back to the join page shows "You already have token A-012". The secret is needed to leave the line, rate the visit, or follow the token live, so a person who only knows the link cannot cancel someone else's place.

In the code: frontend/src/hooks/useGuestToken.js:8; backend/src/services/token.service.js (guestSecretHash)

3. Backend (Node.js + Express)

How the API is structured and how a request is processed.

18. Explain the request path: route, middleware, controller, service, model. Why split it?

Each layer has one job:

- **Route**: URL + method, and the list of checks that run first.
- **Middleware**: login, role, tenant, validation, rate limits; any of them can stop the request.
- **Controller**: reads the request, calls one service, sends the response.
- **Service**: the business rules ("keep at least one service", "one active token per phone").
- **Model**: Mongoose schema and database access.

Splitting it means rules live in one place and can be reused: background jobs and Socket.IO code call the same services without HTTP, and tests can target each layer.

```
router.post('/login', loginLimit, validate({ body: loginSchema }), auth.login);
//           URL        rate limit  check the body with Zod         controller
```

In the code: backend/src/routes/auth.routes.js:11

19. Why are your controllers so thin?

Because HTTP is only one way into the system. The no-show job, the Socket.IO handlers and the tests all need the same rules. If rules were in controllers, each of those would duplicate them. A thin controller (read input, call service, return JSON) also makes the API easy to read: for example `createService` in admin.controller.js is three lines.

In the code: backend/src/controllers/admin.controller.js:52

20. What changed in Express 5 that affected your code?

- **Async errors are forwarded automatically**: if an async handler throws, Express passes it to the error middleware, so no try/catch wrappers are needed in routes.
- `req.query` **is read-only**, so the validated query is stored on `req.validatedQuery`.
- `req.body` is undefined when there is no body (handled in the validator).
- The wildcard route `'*'` syntax changed, so the 404 handler is a final `app.use`.

In the code: backend/src/middleware/validate.middleware.js:7, backend/app.js (404 then errorMiddleware)

21. How are errors handled centrally?

Code throws an `ApiError` (with status, code, message, details) or lets a library error bubble up. The last middleware, `errorMiddleware`, converts everything into one JSON shape: `{ ok:false, error:{ code, message, details } }`. It knows Zod validation errors (400 with one entry per field), MongoDB duplicate keys (409), invalid ids (400), file-upload errors, and logs unexpected errors with a stack trace while hiding details from the client in production.

```
if (err instanceof ApiError) return send(res, err.statusCode, err.code, err.message,
err.details);
if (err instanceof z.ZodError) {
  const details = err.issues.map((i) => ({ field: i.path.join('.') || '(root)', message:
i.message }));
  return send(res, 400, 'VALIDATION_ERROR', 'Some fields are not valid', details);
}
if (err.code === 11000) return send(res, 409, 'CONFLICT', 'This already exists', { fields:
Object.keys(err.keyValue ?? {}) });
if (err instanceof mongoose.Error.CastError) return send(res, 400, 'INVALID_ID', `Invalid value
for ${err.path}`);
```

In the code: backend/src/middleware/error.middleware.js:15

22. How do you validate requests? What does strictObject do?

Each route lists Zod schemas for body, params and query; the `validate` middleware parses them, replaces `req.body` with the clean parsed values (trimmed, lower-cased, converted to numbers), and reports all problems at once. `z.strictObject` **rejects unknown fields**, so a client cannot sneak in extra properties such as `plan` or `businessId` on an update.

In the code: backend/src/middleware/validate.middleware.js:7; rules in backend/src/validators/

23. What does tenantScope do?

It runs after login on every owner and staff route. It loads the logged-in user's business from the database and puts it on `req.business`. Every service then queries with that business's id, so the business always comes from the **verified login**, never from what the client sends. If the client sends a `businessId` for another business, it is refused.

```
export async function tenantScope(req, res, next) {
  const business = await Business.findById(req.user.businessId);
  if (!business) throw ApiError.notFound('BUSINESS_NOT_FOUND', 'This business no longer exists');
  // A businessId in the body must match the session's (it is never used to pick the business).
  if (req.body?.businessId !== undefined && String(req.body.businessId) !== String(business._id))
throw ACCESS_DENIED();
  req.business = business;
  req.businessId = business._id;
  next();
}
```

In the code: backend/src/middleware/tenant.middleware.js:12

24. Which HTTP status codes do you use, and when?

Code	Used for
200 / 201	Success / something created (signup, new token)
202	Accepted, work continues in the background (deleting a business)
308	Old public link redirects to the new one
400	Invalid input (with field details); wrong PIN
401	Not logged in / token expired (TOKEN_EXPIRED triggers a refresh)
402	Plan does not include the feature (only when plan limits are on)
403	Logged in but not allowed (wrong role, another business)
404	Not found (also for other businesses' ids)
409	Conflict: email taken, token already called, queue not live
423	Locked after too many wrong PINs
429	Rate limited, with a Retry-After header

25. Why return 404 instead of 403 for another business's record?

A 403 would confirm that the id exists, which leaks information ("this token exists, you just can't see it"). Because every query includes `businessId`, another business's record is simply not found, and the API answers exactly as it would for an id that never existed. The tenant-isolation tests check this for every collection.

In the code: backend/tests/tenantIsolation.test.js

26. What happens when the server shuts down?

On SIGTERM or SIGINT, `server.js` shuts down gracefully: it stops the job timers, closes Socket.IO (disconnecting clients and stopping the HTTP server from accepting new connections), closes Redis and then MongoDB, and exits. A 10-second safety timer forces exit if something hangs. This avoids half-finished writes and lets clients reconnect to another instance cleanly.

In the code: backend/server.js (shutdown function)

27. How are background jobs scheduled, and what stops them running twice on two servers?

Four jobs run on timers inside the API process: auto no-shows (every 30 s), invite reminders (every minute), daily summary (08:00) and nightly cleanup (03:00). With several servers, every instance has the same timers, so each run first **claims a Redis lock** with `SET key value NX EX seconds`: only the first instance gets `OK`, the others skip. Daily jobs claim one key per date, so a server that was down at 03:00 still runs cleanup when it comes back.

```
async function claim(key, seconds) {
  return (await redis.set(`job:${key}`, String(process.pid), 'EX', seconds, 'NX')) === 'OK';
}
```

In the code: backend/src/jobs/index.js:18

4. Database (MongoDB)

Collections, indexes, transactions and data rules.

28. Why MongoDB instead of a SQL database?

A business's configuration is naturally a nested document: opening hours per day, queue settings, a custom join form (a list of fields), branding, notification preferences. In MongoDB that is one document read in one query; in SQL it would be several tables and joins. Mongoose gives schemas and validation on top. MongoDB also has what I needed for correctness: unique and partial indexes, atomic updates, TTL indexes, and multi-document transactions.

A fair answer also names the trade-off: reporting across many relations is easier in SQL, and I had to design indexes carefully for analytics queries.

29. What are your collections and how are they related?

14 collections: `businesses`, `users`, `services`, `resources` (doctors/counters), `tokens`, `queues` (daily number counters), `staffinvites`, `subscriptions`, `invoices`, `notifications`, `activitylogs`, `slughistories` (old links), `webhookevents`, `enterpriseleads`.

Almost every document has a `businessId`; that is the tenant key. References use ObjectIds (a token points to its service and resource; a resource lists the service ids it handles). Only `enterpriseleads` has no business, because a lead is not a customer yet.

In the code: backend/src/models/*.js

30. How do you guarantee one phone cannot hold two active tokens?

The database enforces it. Active tokens (waiting, called, serving) have a field `activePhone`; finished tokens have it removed. A **unique partial index** on `businessId + activePhone` means MongoDB itself refuses a second active token for the same phone at the same business, even if two requests arrive at exactly the same moment. The service catches that duplicate-key error and returns the existing token instead of failing.

```
// One active token per phone per business, enforced by the database even for simultaneous joins.
tokenSchema.index({ businessId: 1, activePhone: 1 },
  { unique: true, partialFilterExpression: { activePhone: { $exists: true } } });
```

In the code: backend/src/models/Token.js:56

31. How are token numbers generated so two customers never get the same number?

Each business has one counter document per day in the `queues` collection. Getting a number is a single atomic `findOneAndUpdate` with `$inc`: MongoDB increments and returns the new value in one step, so two simultaneous requests always receive different numbers. `upsert: true` creates the day's document on the first token. The same update also increments `total`, which is used for the daily limit.

```
const filter = { businessId, queueKey: 'all', date };
const update = { $inc: { [`counters.${prefix}`]: 1, 'counters.total': 1 } };
const doc = await Queue.findOneAndUpdate(filter, update, { upsert: true, returnDocument: 'after',
session });
const number = doc.counters.get(prefix);
```

In the code: backend/src/utils/tokenNumber.js:22

32. Why does MongoDB need to run as a replica set here?

Multi-document **transactions** only work on a replica set (or sharded cluster). Locally, `ql-mongo` runs as a single-node replica set called `rs0`; in tests, mongodb-memory-server starts a replica set too. Atlas clusters are replica sets by default.

33. Where did you use transactions, and why?

Wherever several writes must succeed or fail together:

- **Signup**: business, owner, template services and resources are created together.
- **Joining the queue**: take a number, check the daily limit, create the token. If the limit is exceeded, the transaction aborts and the number is not used up.
- **Accepting an invite**: claim the invite and create the user together.
- **Stripe webhooks**: record the event id and apply the change together (idempotency).

```
await mongoose.connection.transaction(async (session) => {
  const { number, code, date, total } = await nextTokenNumber({ businessId: business._id, prefix,
timeZone: business.timeZone, session });
  if (total > limit) throw ApiError.conflict(ERROR_CODES.LIMIT, "Today's line is full. ...", {
limit });
  [token] = await Token.create([{ businessId: business._id, queueKey, number, code, day: date, /*
... */ }], { session });
});
```

In the code: backend/src/services/token.service.js:131

34. What indexes did you create and why?

Indexes follow the actual queries:

- `tokens`: `{businessId, status, queueKey, priority, sortAt}` for "who is next in this line", `{businessId, createdAt}` for history and analytics, `{businessId, day}` for daily totals, and the unique `activePhone` index.
- `users.email` unique; `businesses.slug` unique; `queues {businessId, queueKey, date}` unique.
- TTL indexes on activity logs, old slugs and webhook events.

Every query starts with `businessId`, so indexes lead with it.

In the code: backend/src/models/Token.js:48-58

35. What is a TTL index, and where did you use one?

A TTL (time-to-live) index tells MongoDB to delete documents automatically after a time. I used them for **activity logs** (180 days), **old public links** (deleted when `redirectUntil` passes) and **Stripe webhook records** (90 days). No cron job is needed for these.

In the code: backend/src/models/ActivityLog.js, SlugHistory.js, WebhookEvent.js

36. Why are amounts stored in paise?

Floating-point numbers cannot represent many decimals exactly (0.1 + 0.2 is not 0.3), so money is stored as whole numbers of the smallest unit: ₹999 is stored as 99900. This is also how Stripe expects amounts.

In the code: backend/src/models/Invoice.js

37. How do you handle dates and time zones?

MongoDB stores timestamps in UTC. Things that depend on "today" (the daily token counter, analytics days, the nightly cleanup) use the **business's** time zone (Asia/Kolkata by default) through a helper `dateKey()` that produces `YYYY-MM-DD`. Each token stores its local `day`, so "today's tokens" is a simple indexed query, and a business in another time zone rolls over at its own midnight. A test checks a London business and an Indian one at the same instant.

In the code: backend/src/utils/tokenNumber.js (dateKey); backend/tests/jobs.test.js

38. How would you handle a collection with millions of tokens?

Queries are already indexed by business and time and paginated (Customers uses page and pageSize, CSV export streams with a cursor so memory stays flat). Next steps would be: archiving old tokens to a separate collection or cold storage, pre-computing daily analytics into a summary collection instead of aggregating raw tokens every time, and, at very large scale, sharding by `businessId`.

In the code: backend/src/services/customers.service.js:78 (streamed CSV)

5. Authentication and security

The most common deep-dive area. Know these answers well.

39. Explain your login flow end to end.

1. The Login page sends `POST /api/auth/login` with email and password.
2. A rate limiter counts failed attempts per IP and per email (Redis).
3. The service finds the user and compares the password with the stored bcrypt hash. Unknown email and wrong password give the **same** message, and a dummy hash is compared when the user does not exist, so timing does not reveal which emails exist.
4. The server creates a 15-minute **access token** (JWT) and a 30-day **refresh token**; the refresh token is set as an HTTP-only cookie.
5. The frontend keeps the access token in memory, loads `/api/auth/me`, and opens Socket.IO with the token.

```
export async function login({ email, password }) {
  const user = await User.findOne({ email });
  const passwordOk = await bcrypt.compare(password, user?.passwordHash ?? DUMMY_HASH);
  // Same answer for "no such user" and "wrong password", so nobody can probe which emails exist.
  if (!user || !passwordOk) throw ApiError.unauthorized('INVALID_LOGIN', INVALID_LOGIN);
  if (user.status !== 'active') throw ApiError.forbidden('ACCOUNT_DEACTIVATED', DEACTIVATED);
  user.lastLoginAt = new Date();
  await user.save();
  return user;
}
```

In the code: backend/src/services/auth.service.js:46

40. Why two tokens, an access token and a refresh token?

A JWT cannot be "taken back" once issued, so it should be short-lived: the access token lasts 15 minutes and is sent on every request. If it leaks, the damage window is small. The refresh token lasts 30 days but is only sent to `/api/auth` routes, inside an HTTP-only cookie that JavaScript cannot read; it is used only to get new access tokens. This gives security (short exposure) and convenience (users stay logged in).

In the code: backend/src/services/auth.service.js:25 (issueTokens)

41. JWTs are stateless. How does logout actually log someone out?

Each user has a `refreshTokenVersion` number in MongoDB, and the refresh token carries the version it was made with. Logout (and password reset, and deactivation) **increases the version**. On the next refresh, the versions no longer match and the server refuses, so every old session ends. Access tokens still work for at most 15 minutes, but `requireAuth` also reloads the user on every request, so a deactivated user is refused immediately.

```
// refresh: the version in the cookie must still match the user's current version
if (!user || user.status !== 'active' || payload.v !== user.refreshTokenVersion) {
  throw ApiError.unauthorized('SESSION_EXPIRED', 'Your session has ended. Please log in again.');
}
// logout: every refresh token made before this stops working
await User.updateOne({ _id: payload.sub }, { $inc: { refreshTokenVersion: 1 } });
```

In the code: backend/src/services/auth.service.js:71 (refresh), :81 (logout)

42. How does requireAuth check a request?

It reads `Authorization: Bearer <token>`, verifies the JWT signature and expiry with the secret (allowing only HS256), and then **loads the user from the database**. If the token expired, it answers `401 TOKEN_EXPIRED`, which the frontend turns into an automatic refresh. Loading the user fresh means role changes and deactivation take effect at once.

```
payload = jwt.verify(token, env.JWT_ACCESS_SECRET, { algorithms: ['HS256'] });
// ...
const user = await User.findById(payload.sub);
if (!user || user.status !== 'active') throw ApiError.unauthorized('UNAUTHORIZED', 'Please log in
to continue');
req.user = user;
```

In the code: backend/src/middleware/auth.middleware.js:9

43. How are passwords stored? Why bcrypt?

Only a bcrypt hash (cost 12) is stored, never the password. bcrypt adds a random salt per password, so identical passwords produce different hashes and precomputed tables do not work, and it is deliberately slow, which makes guessing millions of passwords expensive. Staff PINs are hashed the same way.

In the code: backend/src/services/auth.service.js (BCRYPT_COST = 12)

44. Why are invite links and reset links hashed in the database?

The link contains a random token. Only its SHA-256 hash is stored. If the database ever leaked, the attacker would see hashes that cannot be turned back into working links. When someone opens the link, the server hashes the token from the URL and looks up the hash. SHA-256 is enough here (no bcrypt needed) because the tokens are long random values, not guessable passwords.

In the code: backend/src/services/invite.service.js, auth.service.js:92 (requestPasswordReset)

45. How can two people never accept the same invite link at the same time?

Accepting is an atomic claim inside a transaction: the invite is updated to `accepted` only if it is still `pending` and not expired. Only one request can win that update; the other gets nothing and receives "this link is no longer valid". The user account is created in the same transaction.

```
const claimed = await StaffInvite.findOneAndUpdate(
  { _id: invite._id, status: 'pending', expiresAt: { $gt: new Date() } },
  { $set: { status: 'accepted', acceptedAt: new Date() } },
  { session, returnDocument: 'after' },
);
if (!claimed) throw INVALID_LINK();
```

In the code: backend/src/services/invite.service.js:238

46. How do you stop brute-force attacks on login and PINs?

Login: Redis-backed rate limits, 10 failed attempts per 15 minutes per IP **and** per email; successful logins do not count, so real users are never blocked. **PIN switching**: wrong PINs are counted in Redis with `INCR` (atomic, so simultaneous guesses all count); after 5 wrong PINs, switching is locked for 30 seconds. Using Redis means the limits hold across several servers.

```
const failures = await redis.incr(pinFailKey(business._id));
if (failures === 1) await redis.expire(pinFailKey(business._id), FAIL_WINDOW_SECONDS);
if (failures >= PIN_MAX_ATTEMPTS) {
  await redis.multi().set(pinLockKey(business._id), '1', 'EX',
PIN_LOCK_SECONDS).del(pinFailKey(business._id)).exec();
  throw ApiError.locked(ERROR_CODES.LOCKED, `Too many wrong PINs. Try again in
${PIN_LOCK_SECONDS} seconds.`);
}
```

In the code: backend/src/middleware/rateLimit.middleware.js:69; backend/src/services/staff.service.js:137

47. What is multi-tenancy, and how do you guarantee isolation?

Multi-tenancy means many businesses (tenants) share one application and one database. Isolation is enforced in layers: every document carries a `businessId`; `tenantScope` takes the business from the verified login only; every query includes that id; Socket.IO rooms are per business and joined from the login, not from client input. A dedicated test suite tries to read, update and delete each kind of record of business B while logged in as A, and expects 404 every time.

In the code: backend/src/middleware/tenant.middleware.js:12; backend/tests/tenantIsolation.test.js

48. How do you prevent NoSQL injection?

NoSQL injection happens when a client sends an object such as `{"$ne": null}` where a string is expected, turning a lookup into "match anything". Every request is parsed by Zod first, so an email must be a string and an id must be a 24-character hex string; objects are rejected before they reach a query. Express 5's default query parser also does not turn `?a[$ne]=1` into nested objects.

49. Explain the HttpOnly, Secure and SameSite settings on your cookie.

- **HttpOnly**: JavaScript cannot read the cookie, so XSS cannot steal the refresh token.
- **Secure** (in production): only sent over HTTPS.
- **SameSite=Lax**: not sent on cross-site POSTs from other websites, which protects against CSRF.
- **Path=/api/auth**: sent only to refresh and logout, not with every API request.

```
const cookieOptions = { httpOnly: true, secure: env.NODE_ENV === 'production', sameSite: 'lax',
path: '/api/auth' };
```

In the code: backend/src/controllers/auth.controller.js:9

50. What is CORS, and how did you configure it?

Browsers block a page from reading responses from another origin unless the server allows it. The API allows exactly one origin, `CLIENT_URL` (http://localhost:5173 locally), with `credentials: true` so the refresh cookie can be sent. Any other website calling the API from a browser is blocked.

In the code: backend/app.js:39

51. What does helmet do?

It sets security-related HTTP headers in one line: Content-Security-Policy, Strict-Transport-Security (HTTPS only), X-Content-Type-Options (no MIME sniffing), frame protection against clickjacking, and others. In production the app also redirects plain HTTP to HTTPS.

In the code: backend/app.js:38

52. Why does 'Forgot password' show the same message for unknown emails?

If it said "no account with this email", anyone could use the form to find out who has an account (account enumeration). It always says "If an account exists, a reset link is on its way", and only sends an email when the account exists. Login does the same for wrong email vs wrong password.

In the code: backend/src/services/auth.service.js:92

53. What is CSV formula injection, and how did you prevent it?

A customer could type a name like `=HYPERLINK("http://evil...")`. When the owner opens the exported CSV in Excel, that cell would run as a formula. Every cell that starts with `=`, `+`, `-` or `@` is prefixed with a quote so spreadsheets show it as text; values with commas or quotes are quoted properly.

```
export function csvCell(value) {
  if (value == null) return '';
  let text = String(value);
  if (/^[=+\-@\t\r]/.test(text)) text = `'${text}`;
  return /[",\r\n]/.test(text) ? `"${text.replace(/"/g, '""')}"` : text;
}
```

In the code: backend/src/services/customers.service.js:69

54. How are uploaded logos kept safe?

Uploads go through multer into memory with a 400 KB limit. The server checks the file's first bytes (the "magic number") to confirm it is really PNG, JPEG or WebP, rather than trusting the file name or the browser. SVG is refused because it can contain scripts. Files get a new random name on every upload.

In the code: backend/src/services/branding.service.js (detectImageType)

6. Real-time (Socket.IO and Redis)

How live updates reach exactly the right screens.

55. How do live updates work? Why Socket.IO and not polling?

Each screen keeps one WebSocket connection (Socket.IO). After a database change, the server emits an event to a room, and screens in that room update within about 0.3 seconds. Polling (asking every few seconds) would either be slow or flood the server with requests from every phone, TV and console. Socket.IO adds rooms, automatic reconnection, and fallback transports on top of raw WebSockets. As a safety net, the token page also checks every 30 seconds and the TV every 60 seconds.

56. What are rooms, and which rooms do you have?

A room is a named group of connections; emitting to a room reaches only those connections.

Room	Who	Gets
`business:<id>`	owner and staff screens	full token details
`user:<id>`	all screens of one person	used to disconnect a deactivated user
`public:<id>`	join pages	counts only
`token:<id>`	one customer's phone	their own token only
`display:<id>`	TV screens	codes and counts only

Rooms use ids, not slugs, so renaming a business's link does not break open screens.

In the code: backend/src/sockets/rooms.js:5

57. How do you make sure the TV never receives customer names?

Every outgoing event is built per audience in one function, `broadcast()`. The staff room gets the full token (with the customer); the TV and the customer's phone get only the code and the counter name. A test listens on a join page and a TV and asserts that no name or phone ever appears in what they receive.

```
case 'token:called': {
  // The TV chime and flash, and the "It is your turn" alert. Code and counter only.
  const called = { code: token.code, resource: { id: String(resource._id), name: resource.name }
};
  emitTo(rooms.display(business._id), 'token:called', called);
  emitTo(rooms.token(token._id), 'token:called', called);
  break;
}
```

In the code: backend/src/sockets/queue.socket.js:117

58. How does a customer's phone join only its own token room?

The phone sends `token:join` with the slug, the token id and its guest secret. The server finds the token, compares the secret with the stored hash, and only then adds the socket to `token:<id>`. Without the secret, nobody can follow someone else's token. A guest connection can join at most 10 rooms, so one connection cannot subscribe to everything.

In the code: backend/src/sockets/queue.socket.js (token:join handler)

59. How does the server know who a socket belongs to?

Owner and staff screens send their access token in the Socket.IO handshake (`auth: { token }`). The handshake middleware verifies it and loads the user from the database, then the socket joins that user's business room automatically. The business always comes from the user record, never from the client. Guests (phones, TVs) connect without a token and must ask for public rooms.

In the code: backend/src/sockets/auth.socket.js:13

60. What happens with two backend servers? How do events reach clients on the other one?

Each server only knows its own connections. The **Redis adapter** publishes every emit to a Redis channel; all servers subscribe and deliver it to their local clients in that room. So staff on server A pressing Call next still updates a TV connected to server B. A test starts two Socket.IO servers sharing Redis and checks this.

```
io = new Server(httpServer, socketOptions);
if (pubClient && subClient) io.adapter(createAdapter(pubClient, subClient));
```

In the code: backend/src/sockets/index.js:14

61. What happens if a phone's connection drops?

Socket.IO reconnects automatically. Rooms are forgotten on reconnect, so the pages rejoin their rooms in a `connect` handler and reload their data to catch up on anything missed. The periodic backup check covers longer outages. A logged-in socket reconnects with the newest access token because the handshake reads it through a function each time.

In the code: frontend/src/lib/socket.js; frontend/src/pages/customer/TokenStatus.jsx

62. Why reload data on an event instead of trusting the event's payload?

The server is the single source of truth. Reloading guarantees every screen shows exactly what the database says, even if events arrive out of order or one was missed during a reconnect. Events stay small and safe (no private data needed in them), and the reload is debounced so a burst of events triggers one request.

7. Concurrency and system design

Interviewers love these. Use the words 'atomic', 'race condition', and 'single source of truth'.

63. Two staff press 'Call next' at the same moment. What happens?

Both may read the same first person in line. But the call itself is one atomic update that only matches while that token is still `WAITING`. MongoDB applies the two updates one after the other: the first changes it to `CALLED`, the second matches nothing and gets `null`, so it moves on to the next person. Two staff can never call the same customer. A test runs both calls in parallel and checks they got different tokens.

```
for (let attempt = 0; attempt < CLAIM_ATTEMPTS; attempt += 1) {
  const candidate = await nextFor(business, resource);
  if (!candidate) break;
  const called = await Token.findOneAndUpdate(
    { _id: candidate._id, businessId: business._id, status: WAITING },   // only if still waiting
    { $set: { status: CALLED, resourceId: resource._id, calledAt: new Date() } },
    { returnDocument: 'after' },
  );
  if (called) { /* emit events */ return { called, finished }; }
}
```

In the code: backend/src/services/staff.service.js:55

64. A customer taps 'Get my token' five times quickly. What happens?

On the frontend the button is disabled while submitting. On the server, all five requests may run in parallel, but the unique `activePhone` index lets only one token be created; the others hit a duplicate-key error, which the service catches and answers with the existing token. The test "five taps at the same instant create one token" checks it.

In the code: backend/src/services/token.service.js (catch err.code === 11000)

65. Many people join at the same time right at the daily limit. Can extra tokens slip through?

No. The daily total is incremented atomically together with the token number inside the same transaction, and the limit is checked against that freshly incremented total. If it is over the limit, the transaction aborts, so nothing is saved and the number is not used up.

In the code: backend/src/services/token.service.js:131

66. How would you scale QueueLess to 10,000 businesses?

- The API is **stateless** (JWT, no server sessions), so run several instances behind a load balancer.
- Socket.IO already scales across instances through the **Redis adapter**; rate limits and job locks are already in Redis.
- **MongoDB**: a managed replica set, the existing indexes, pre-computed daily analytics, and sharding by `businessId` if needed.
- Move timed jobs and SMS/email sending to a **queue with workers** (BullMQ).
- Serve the frontend from a CDN.

67. Where would the bottleneck be under heavy load?

Likely the analytics aggregations over raw tokens (they scan many documents) and the per-line "send positions to everyone waiting" step after each change on very long lines. Fixes: cache or pre-aggregate analytics daily, and throttle or batch position updates per line.

68. Why Redis for rate limiting instead of memory?

An in-memory counter lives in one process. With three servers, an attacker would get three times the allowed attempts, and a restart would reset the counts. Redis is shared by all instances and atomic, so limits hold everywhere. If Redis is briefly unavailable the limiter lets requests through (fail open) rather than locking everyone out.

In the code: backend/src/middleware/rateLimit.middleware.js

69. How does the 'almost your turn' alert fire exactly once?

When a line moves, the server looks at the first few waiting tokens. For each, it does an atomic claim: set `alertedAlmostAt` only if it is still null. If two line changes happen at once, only one claim succeeds, so the customer gets one alert, not two.

```
const claimed = await Token.findOneAndUpdate(
  { _id: token._id, status: TOKEN_STATUS.WAITING, alertedAlmostAt: null },
  { $set: { alertedAlmostAt: new Date() } });
if (!claimed) continue;
emitTo(rooms.token(token._id), 'token:almost', { code: token.code, ahead });
```

In the code: backend/src/services/notification.service.js:195

70. Stripe sends the same webhook twice. What happens?

Each event id is saved in `webhookevents` inside the same transaction as the change (new plan, new invoice), with a unique index on the event id. A repeat hits the unique index and is skipped; a failure saves nothing, so Stripe's automatic retry runs it again cleanly. This is called **idempotency**.

In the code: backend/src/services/billing.service.js:331

71. How would you deploy this? Why can the backend not run on Vercel?

The frontend is static files, perfect for Vercel. The backend must stay running: it holds open WebSocket connections and runs timed jobs, while Vercel runs code as short serverless functions that stop after each request. So the backend goes on a host with long-running processes (Render, Railway, AWS ECS), with MongoDB Atlas and a cloud Redis (Upstash). Because the frontend and API would be on different sites, the refresh cookie needs care: proxy `/api` through the frontend's domain or use a shared custom domain.

8. Features and business logic

Questions about how specific features work.

72. Explain the three queue modes.

- **Shared** (canteen): one line; any counter calls the next person.
- **Per service** (bank): customers choose a service; a counter calls people waiting for the services it handles.
- **Per resource** (clinic): customers choose a doctor and wait in that doctor's line, optionally "any available".

The mode decides each token's `queueKey` (`shared`, `svc:<id>`, `res:<id>`); "who is next" and "people ahead" are queries on that key, ordered by priority, then arrival.

In the code: backend/src/services/queue.service.js:19 and :39 (QUEUE_ORDER)

73. How is the wait time estimated?

`round((people ahead + 0.5) × average minutes per service ÷ number of counters serving that line)`. The 0.5 accounts for the person currently being served being part-way through. A comment in the code notes the next improvement: use a rolling average of real service times instead of the configured average.

```
export const estimateMinutes = ({ ahead, avgMinutes = DEFAULT_AVG_MINUTES, servers = 1 }) =>
  Math.round(((ahead + 0.5) * avgMinutes) / Math.max(1, servers));
```

In the code: backend/src/services/eta.service.js:20

74. What happens to people still waiting when the day ends?

The nightly cleanup job (03:00) cancels tokens still waiting, called or serving from earlier days (in each business's own time zone), marks them cancelled by the system, and removes `activePhone` so those phones can join again the next day. It uses one `updateMany` with an update pipeline per business.

In the code: backend/src/jobs/cleanup.job.js:13

75. How do public links (slugs) and old-link redirects work?

Each business has a unique slug (`city-care`), created from its name at signup. If a live business changes its link, the old slug is saved in `slughistories` with an expiry 90 days ahead, and requests to the old slug answer `308` with the new address, so printed QR codes keep working for a while. MongoDB deletes the history row when it expires. Reserved words (admin, login, ...) cannot be slugs.

In the code: backend/src/services/slug.service.js:23 (resolvePublicSlug), :58 (changeSlug)

76. Walk me through the staff invite flow.

1. The owner enters an email, role and counter. The server creates an invite with a random 32-character token (stored hashed) and emails the link (Nodemailer via Gmail SMTP).
2. The invitee opens `/accept-invite/<token>`; the page shows the business and role.
3. They set a name, password and optional PIN. The server claims the invite atomically and creates the account in one transaction, then logs them in.
4. Resend creates a new token, so the old link stops working; revoke cancels it; links expire after 7 days.

In the code: backend/src/services/invite.service.js:238

77. How does 'Switch user' with a PIN work on a shared computer?

The front desk shows staff names (no emails). Picking a person and typing their 4-digit PIN calls `/api/staff/switch-user`; the server checks the PIN against the bcrypt hash and, if correct, issues new tokens for that person, so the console switches user without a full login. Wrong PINs are counted in Redis and lock switching for 30 seconds after five.

In the code: backend/src/services/staff.service.js:137

78. How do you keep customer data private?

The join form asks only what the business configures (name and phone always). Phones are removed from tokens older than 90 days by the nightly job, while the rest of the history stays for analytics. Public pages and TVs never receive names or phones. Customer secrets are stored hashed. Deleting a business removes its data (invoices are kept for tax records).

In the code: backend/src/jobs/cleanup.job.js:31

79. How does analytics compute 'change vs yesterday'?

Comparing all of today with all of yesterday at 10 am would always look bad. So the server compares today so far with yesterday **up to the same time of day**, using MongoDB aggregation pipelines that count issued, served, no-shows and average wait. Hours are grouped in the business's time zone.

In the code: backend/src/services/analytics.service.js:32 (daySummary)

80. Why did you remove the plan limits? How?

The project has no real payments, so plan restrictions only got in the way. Instead of deleting the enforcement code, all checks go through two functions, `hasFeature()` and `planLimit()`, controlled by one setting, `PLAN_LIMITS` (default off). Off means every feature is available with no limits; the existing tests switch it on to keep the enforcement code tested, and a separate test checks the off behaviour.

```
export const PLAN_LIMITS_ON = env.PLAN_LIMITS === 'on';
export const hasFeature = (businessOrPlanId, feature) => !PLAN_LIMITS_ON || Boolean(/* the plan's
features */);
export const planLimit = (planId, key) => (PLAN_LIMITS_ON ? getPlan(planId).limits[key] :
Infinity);
```

In the code: backend/src/utils/constants.js:122

81. How does deleting a business work safely?

The owner must type the business's link to confirm. The server first cancels any paid subscription (if that fails, nothing is deleted), copies billing details onto invoices (kept for tax records), deletes the business and its users at once so every login and screen stops, disconnects their sockets, and deletes the remaining data in the background. The API answers 202 Accepted.

In the code: backend/src/services/business.service.js:307

9. Testing and debugging

Show that you can prove your code works and find problems methodically.

82. How did you test the backend?

17 test files, 247 tests, using **Jest** and **Supertest** (which calls the Express app directly, without opening a port), against a real MongoDB started in memory by **mongodb-memory-server** (as a replica set, so transactions work). Tests cover auth, tenant isolation, slugs, joining, queue order and races, invites and PIN lockout, Socket.IO rooms, billing webhooks, analytics, jobs, rate limits, security and the seed data.

In the code: backend/tests/*.test.js; run with npm test

83. Why an in-memory MongoDB instead of mocking the database?

Many important rules live in the database itself: unique indexes, atomic updates, transactions, aggregation pipelines. A mock would not enforce any of them, so tests could pass while the real behaviour was broken. An in-memory MongoDB is real MongoDB, fast and isolated, and each test file gets a clean one.

84. How do you test a race condition?

Fire the requests at the same time with `Promise.all` and check the invariant. Examples: 20 joins at once must get 20 different numbers; 5 joins from the same phone must create one token; two Call next requests on the same line must call different customers; simultaneous joins at the daily limit must not exceed it.

In the code: backend/tests/token.test.js, queue.test.js

85. How do you test tenant isolation?

Create two businesses, A and B, with records of every kind. Logged in as A: list endpoints must return only A's records; reading, updating or deleting B's record by id must return 404; and a request that names B in the body or query must be refused. A control check confirms B can still read its own record, so the 404s are not caused by a broken route.

In the code: backend/tests/tenantIsolation.test.js

86. How do you test Socket.IO?

The test starts the app with a real Socket.IO server on a random port, connects socket.io-client clients as a staff member, a join page, a TV and a phone, performs actions over HTTP, and waits for the expected events, checking who received what (and who must not). One test runs two servers with the Redis adapter to check cross-instance delivery.

In the code: backend/tests/sockets.test.js

87. What is CI, and what does yours do?

Continuous Integration runs checks automatically on every push. A GitHub Actions workflow starts Node 22 and a Redis service, installs dependencies with `npm ci` and runs the whole test suite. A broken change shows a red cross on GitHub before anyone relies on it.

In the code: .github/workflows/backend-tests.yml

88. Tell me about a real bug you found and fixed.

Pick one and tell it as: symptom, cause, fix, how you verified it.

- **TV stopped updating**: at start-up, the "is anyone logged in?" check failed for visitors and its cleanup closed the shared socket, so the TV went silent. Fix: only close the connection when someone was actually logged in. Verified in a browser: the TV updated 316 ms after Call next.
- **Walk-ins refused**: the server required every join-form field (like Age), but the staff dialog only asks name and phone. Fix: extra fields optional for staff walk-ins, still required online.
- **Fields lost**: spreading a Mongoose subdocument (`{...field}`) dropped its values; fix: convert with `toObject()` first.

89. A screen is not updating live. How would you debug it?

1. DevTools → Network → filter `socket.io`: is the websocket connected? Do event messages arrive?
2. If no events: did the page join its room (e.g. `token:join` answered ok)? Is the backend emitting (check the service calls `emitQueueEvent`)?
3. If events arrive but the screen does not change: is the listener registered, and does it reload the data?
4. Check the backend terminal for errors in "Live update ... failed".

90. A user is logged out after every refresh. Where do you look?

The refresh path: DevTools → Application → Cookies (is `ql_refresh` there?); Network → the `/api/auth/refresh` call (status and error code); CORS must allow credentials from `CLIENT_URL`; the cookie must reach the API's domain (a problem when frontend and API are on different sites); and in Compass, compare `refreshTokenVersion` (a logout elsewhere increases it).

10. Behavioural and resume questions

There is no code answer here; prepare your own honest story using these pointers.

91. Did you build this alone? Did you use AI tools?

Answer honestly and specifically. A strong answer: say which tools you used and how (for example, to generate first drafts or explain errors), and then show ownership: you followed a phased plan, ran and tested every step, and can explain each design decision. Interviewers mainly check whether you understand the code. Being able to answer the questions in this document is the proof.

92. How did you plan the work?

In phases, each with a "done when" checklist: data models → auth → business setup → services and resources → joining and token engine → staff actions → real-time → invites and PINs → billing → notifications, analytics and jobs → security, seed data and tests → connecting the frontend area by area. Each phase built on tested earlier ones.

93. If you started again, what would you do differently?

Good honest points: write frontend tests from the start; design the frontend data layer around the real API from day one instead of a simulated backend; decide earlier on file storage for uploads; and add a job queue rather than timers.

94. How did you keep the code maintainable?

Layered structure with one job per layer; business rules in services; one API client and one error shape; validation schemas in one folder; constants (plans, limits, statuses) in one file; comments that explain *why*; automated tests; and switches (like `PLAN_LIMITS`) instead of deleting code.

95. Which trade-off are you least happy with?

Examples: logos stored on local disk (not suitable for multiple servers); the frontend reloads whole views on events instead of applying small updates (simple and correct, but more requests); wait estimates use configured averages rather than real data.

96. Why these technologies over alternatives?

Choice	Instead of	Why
MongoDB	PostgreSQL	Nested business configuration; flexible join forms; transactions still available
Socket.IO	Raw WebSocket / SSE	Rooms, reconnection, Redis adapter for multiple servers
Express 5	NestJS / Fastify	Simple, well known, automatic async error handling
React + Vite	Next.js	A client-side app is enough; no server rendering needed; fast dev server
Zustand	Redux	Small global state, little boilerplate
Zod	Joi / express-validator	Same library on frontend and backend; clear errors
Redis	In-memory	Shared counters and locks across servers